DressMe — Module M7 « Catalogue local »

Rapport technique · BO6 Marché tunisien · ESPRIT 2026 Collecte des données (scraping), nettoyage, et moteur de recherche de vêtements dans les boutiques tunisiennes.

Mis à jour le 14/10/2026 : 15 boutiques, 2 471 produits.

Sommaire

1. Contexte et objectif
2. Vue d'ensemble
3. Collecte des données : le scraping
4. Nettoyage des données
5. Le module M7
6. Comment tester le module
7. Limites et pistes d'amélioration
8. Annexe : fichiers du module

1. Contexte et objectif

DressMe est un assistant de mode personnel. Le module **M7 « Catalogue local »** répond à un besoin précis du marché tunisien : une utilisatrice cherche par exemple *des baskets blanches à Tunis pour un budget de 80 TND*. Aujourd'hui elle doit parcourir des dizaines de pages Instagram et de sites. M7 lui propose directement des articles de boutiques tunisiennes adaptés à son style, à son budget et à sa ville.

Exemple de résultat obtenu par le module :

> **Manuella · Tunis · Basket à plateforme femme · 69,3 TND · lien vers la fiche produit**

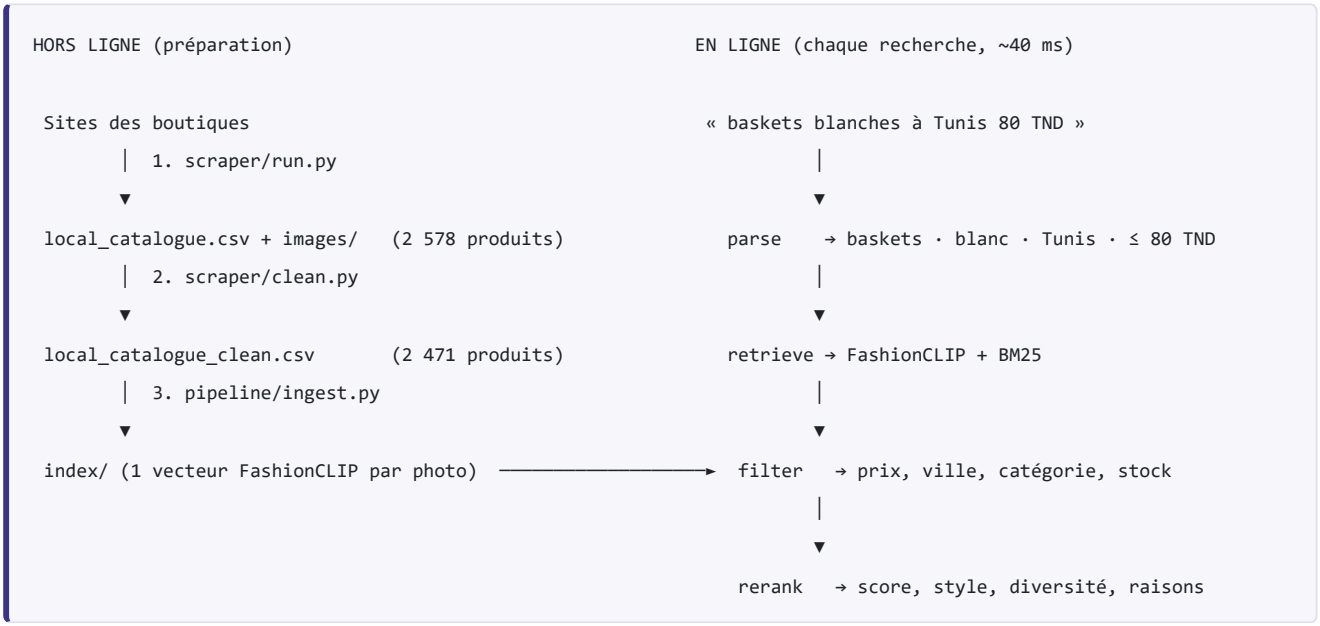
Place dans le projet. D'après le document de répartition des tâches (*Task Split*), M7 est un **module partagé** : il est construit une seule fois par BO6 et appelé par d'autres BO (BO1, BO2, BO3, BO5, BO7, BO8, BO9). Il doit donc exposer une API stable. Le document d'architecture (*DressMe_MultiAgent_Pipeline.pdf*, section 7 « Local shop agent ») fixe :

Élément	Exigence de l'architecture	Ce qui a été réalisé
Base de connaissances	CSV/JSON de boutiques et de pièces, sans scraping en direct	CSV nettoyé de 2 471 produits + index d'images, dans `ml/data/shops/` (DVC)
Recherche	Embeddings + BM25 (hybride)	FashionCLIP (image + texte) + BM25
Filtres	Prix (TND), ville, catégorie, style	Prix, ville/gouvernorat, catégorie, stock, vecteur de style
Intégration	Outil appelable par l'orchestrateur (LangGraph)	Outil `localshop` + API REST `/m07/*` + écran BO6 Flutter
Entrée / sortie	`query, gap, style_vector` → `shop, area, item, price_tnd, match`	Mêmes champs, enrichis (liens, photo, raisons)

La demande complémentaire était d'utiliser **FashionCLIP en local en Python**, ce qui a été fait (§5.2). Ce choix a ensuite été comparé à deux modèles plus récents sur notre catalogue (§5.10).

2. Vue d'ensemble

Le travail se fait en deux temps : une préparation des données **hors ligne** (à relancer de temps en temps pour mettre à jour prix et stocks), puis une recherche **en ligne** très rapide.



Toutes les données (CSV, photos, index) sont dans `ml/data/shops/`. Comme toutes les données du projet, elles ne sont pas dans git mais partagées avec DVC (`ml/README.md`).

2.1 Input → modèle → output

Chaque étape reçoit une entrée (*input*), la transforme avec un modèle ou une méthode, et produit une sortie (*output*) qui devient l'entrée de l'étape suivante.

Étape	Input	Modèle / méthode	Output
1. Scraping	Données d'une boutique (ex. JSON Shopify de Manuella)	Adaptateur de plateforme + règles de normalisation	1 ligne CSV brute + 1 photo JPEG
2. Nettoyage	Ligne brute	Règles, empreinte d'image *dHash*, vérification HTTP	1 ligne propre (26 colonnes)
3. Indexation	Photo JPEG (800 px)	**FashionCLIP — encodeur d'image** (réseau de neurones pré-entraîné)	Vecteur de 512 nombres
4. Compréhension	Phrase de l'utilisatrice (français)	Règles `parse.py`	Critères + phrase en anglais
5. Encodage de la requête	Phrase en anglais	**FashionCLIP — encodeur de texte**	Vecteur de 512 nombres
6. Recherche	Vecteur de la requête + 2 471 vecteurs de photos + mots	Similarité cosinus + BM25	2 scores par produit
7. Filtres et classement	Scores + critères	Filtres, somme pondérée, diversité (MMR)	Les k meilleurs produits, avec `match` et raisons

Seules les étapes 3 et 5 utilisent un modèle d'intelligence artificielle (FashionCLIP). Les autres sont des règles et des calculs : elles sont rapides, gratuites et explicables.

Exemple réel de bout en bout

Le produit suivi est la **« Basket à plateforme femme » de Manuella** ; la requête est celle du sujet.



Hors ligne — le produit entre dans le catalogue

Étape 1 · Scraping. *Input* : la fiche JSON publique de la boutique Shopify (extrait).

```
{"title": "Basket à plateforme femme", "vendor": "Manuella",
 "options": ["couleur", "Pointure"],
 "variants": [{"option1": "Blanc", "option2": "41", "price": "69.300"},
              {"option1": "Blanc", "option2": "40", "price": "69.300"}, "..."]}
```

Output : une ligne brute du fichier `local_catalogue.csv` (et la photo `images/2b7db627f328cf6a.jpg`).

```
Manuella | Basket à plateforme femme | Sneakers | Blanc | 41, 40, 39, 38, 37, 36 | 69.3 TND | Tunis | true
```

Étape 2 · Nettoyage. *Input* : la ligne brute. *Output* : la ligne propre (extrait).

```
{"product_id": "prd_2b7db627f3", "category": "shoes", "subcategory": "sneakers", "label_fr": "Baskets",
 "color": "white", "color_fr": "Blanc", "color_hex": "#FFFFFF",
 "size": "36, 37, 38, 39, 40, 41", "size_type": "numeric", "price_tnd": 69.3, "currency": "TND",
 "city": "Tunis", "governorate": "Tunis", "product_url_status": "ok", "missing_fields": ""}
```

Étape 3 · Indexation (FashionCLIP). *Input* : la photo ci-dessus. *Output* : un vecteur de 512 nombres qui décrit son contenu visuel.

```
[-0.015, 0.046, -0.003, -0.019, -0.045, … 507 autres nombres]
```

En ligne — la recherche de l'utilisatrice

Input de l'API (`POST /m07/search`) :

```
{"query": "baskets blanches à Tunis pour un budget de 80 TND", "k": 3}
```

Étape 4 · Compréhension. *Output* :


```
{"category": "shoes", "subcategory": "sneakers", "color": "white", "max_price_tnd": 80.0,
 "city": "Tunis", "governorate": "Tunis", "terms": ["basket", "blanche"],
 "prompt_en": "a photo of white sneakers", "corrected": null}
```


Étape 5 · Encodage de la requête (FashionCLIP). *Input* : `"a photo of white sneakers"`. *Output* : `[-0.047, 0.004, -0.097, -0.006, 0.003, … 507 autres nombres]`.

Étape 6 · Recherche. Pour notre basket :

- similarité cosinus entre le vecteur de la requête et celui de la photo : **0,289** (le meilleur score du catalogue est 0,342) ;
- score BM25 des mots « basket » et « blanche » dans sa fiche : **3,91**.

Étape 7 · Filtres et classement.

- Filtres (baskets, prix ≤ 80 TND, gouvernorat de Tunis, en stock) : **2 471 → 16 produits**.
- Composantes du score de notre basket :
  - visuel = (0,289 − 0,15) / (0,33 − 0,15) = **0,772** ;
  - texte = **0,974** ;
  - attributs (baskets + blanc) = **1,0** ;
  - pas de vecteur de style dans cette requête.
- `match` = (0,45 × 0,772 + 0,20 × 0,974 + 0,20 × 1,0) / (0,45 + 0,20 + 0,20) = **0,874**.

Output de l'API (extrait, les 3 premiers résultats) :

match	Boutique	Article	Prix	Raisons
0,882	Manuella · Tunis	Basket femme	62,65 TND	Baskets · Autre couleur : gris · 62.65 TND (budget 80 TND) · Boutique à Tunis · Ressemble visuellement à la demande · Marque tunisienne
0,874	**Manuella · Tunis**	**Basket à plateforme femme**	**69,3 TND**	Baskets · Couleur : blanc · 69.3 TND (budget 80 TND) · Boutique à Tunis · Ressemble visuellement à la demande · Marque tunisienne
0,761	Joliesse · Tunis…	Baskets Basses Effet Serpent à Détails Dorés	79,99 TND	Baskets · Autre couleur : doré · 79.99 TND (budget 80 TND) · Boutique à Tunis · Ressemble visuellement à la demande · Marque tunisienne


```
{"shop": "Manuella", "area": "Tunis", "item": "Basket à plateforme femme", "price_tnd": 69.3,
 "match": 0.874, "product_id": "prd_2b7db627f3", "category": "shoes", "subcategory": "sneakers",
 "color": "white", "size": "36, 37, 38, 39, 40, 41", "available": true,
 "image_path": "images/2b7db627f328cf6a.jpg",
 "product_url": "https://manuella.com.tn/products/basket-a-plateforme-femme",
 "reasons": ["Baskets", "Couleur : blanc", "69.3 TND (budget 80 TND)", "Boutique à Tunis",
             "Ressemble visuellement à la demande", "Marque tunisienne"]}
```

3. Collecte des données : le scraping

3.1 Choix des sources

Le point de départ était une liste de **28 sources** : 5 places de marché ou annuaires (Ad. Mall, Moudda, Lofa, Hawwini, Boutique.tn), 7 enseignes de prêt-à-porter (VOG, HA, Apparel, Celina Fashion, MAGIC, UNJA, Corella) et 16 boutiques citées dans l'annuaire Hawwini. **9 sources** ont été ajoutées ensuite, surtout pour les bijoux, accessoires et chaussures (Strass Accessoires, HK-STORE, GAŸA, The Queen Accessories, SKILA, Joliesse, Azar Shoes, Chaussures Pitillos, Shoes Center), soit **37 sources**.

Trouver les vraies adresses. Pour les boutiques de l'annuaire, seules les marques étaient connues. Les adresses des sites ont été retrouvées sur les fiches « retailer » de Hawwini (par exemple NOON → `noonclo.com`, Manuella → `manuella.com.tn`), puis par recherche web pour les ajouts. *Pourquoi :* deviner les domaines aurait donné de faux sites.

Sources écartées, avec la raison (la liste est aussi dans `scraper/shops.py`) :

Source	Raison
JENNAH Boutique, Ayada Boutique	Boutiques **françaises** (prix en euros, adresses à Pantin et Aulnay-sous-Bois)
Scarpa Store, Sneakers Tunisie, Sneakers Outlet	Ne revendent **que des marques étrangères** (Nike, Adidas, Saucony, Givenchy…) : il ne restait rien après le filtre « local uniquement »
Chaussures Pitillos	Marque **espagnole** (Arnedo, calzadospitillos.com) : pas une marque locale
Lofa	Catalogue de démonstration (14 produits avec des photos de banque d'images)
Celina Fashion, SKILA	Sites vitrines : pas de catalogue en ligne ni de prix
Dabchy	Seconde main entre particuliers, pas des boutiques
Hawwini	Comparateur de prix : utilisé seulement pour trouver les adresses
Shoes Center	`shoescenter.tn` est un domaine « parqué » (aucune boutique), `shoescenter.com.tn` a un certificat invalide
HK-STORE, GAŸA, The Queen Accessories, Azar Shoes	Aucun site trouvé (vente en magasin ou sur les réseaux sociaux)
Fripup, MODECO, Spadri, Goat Sneakers	Site en panne (erreur 520), injoignable ou introuvable
Ad. Mall, boutique.tn, D'Stock	Pas de données structurées ; nécessiteraient un analyseur HTML dédié (piste future)

Résultat : 15 boutiques dans le catalogue final.

3.2 Détection de la plateforme de chaque site

Avant d'écrire le moindre code de scraping, chaque site a été analysé : quelle plateforme e-commerce utilise-t-il ? Pour cela, on a examiné la page d'accueil (balise `generator`, signatures comme `cdn.shopify.com` ou `wp-content`) et testé les adresses de données connues de chaque plateforme.

Plateforme	Boutiques	Méthode de récupération
Shopify	Moudda, NOON, Manuella	Flux public `/products.json`
WooCommerce	VOG, Corella, Apparel	API publique *Store API* `?rest_route=/wc/store/v1/products`

Plateforme	Boutiques	Méthode de récupération
PrestaShop 1.7+	Exist, FEModest, Shoes Mall, Strass Accessoires, Joliesse	Pages catégories demandées en JSON (en-tête `X-Requested-With`), tailles lues sur la fiche produit
PrestaShop 1.6	LYOUM	Lecture des « cartes produit » HTML (thème personnalisé)
Pages avec JSON-LD	MAGIC, UNJA	Données `schema.org/Product` présentes dans chaque fiche produit
API privée	HA	API JSON utilisée par leur propre site (`/api/items/list`)

Pourquoi cette approche : les plateformes exposent déjà leurs produits sous forme de **données structurées** (JSON). C'est beaucoup plus fiable que d'extraire du texte depuis le HTML : le prix, la devise, les variantes (couleurs, tailles), le stock et les photos arrivent déjà séparés. On a écrit **un adaptateur par plateforme** (`scraper/sources.py`) plutôt qu'un code par site : Strass et Joliesse ont été ajoutées en quelques lignes de configuration. Seul le thème de Joliesse (Creative Elements) a demandé d'adapter la lecture des pointures.

3.3 Un scraping respectueux des sites

Règle	Mise en œuvre	Raison
Respect du `robots.txt`	Chaque URL est vérifiée avant d'être demandée	Les sites indiquent ce qu'ils autorisent
Pas de surcharge	0,5 s minimum entre deux requêtes vers un même site (1 s pour la vérification des URL)	Ne pas gêner les boutiques
Robot identifiable	User-Agent `DressMeCatalogueBot/0.1 (student project, ESPRIT)`	Transparence
Erreurs temporaires	Jusqu'à 6 nouvelles tentatives avec attente croissante (429, 5xx)	Robustesse
Pas de cookies	Client « sans état »	Voir ci-dessous

Problème rencontré : erreurs 429 sur Shopify. Les boutiques Shopify répondaient « 429 Too Many Requests » alors qu'une simple requête fonctionnait. L'analyse a montré que Shopify bloque le flux `/products.json` **dès que ses cookies de session sont présents** (posés lors de la lecture du `robots.txt`). Solution : un client qui refuse tous les cookies, dont les sites n'ont pas besoin pour afficher un catalogue public.

3.4 Normalisation pendant la collecte

Chaque boutique décrit ses produits à sa façon. Pour obtenir un catalogue homogène, on applique des règles par mots-clés (`scraper/normalize.py`) :

- **Catégorie** à partir du nom du produit, puis de la catégorie de la boutique. Les règles couvrent le français, l'anglais, les mots tunisiens et l'arabe : *dengri* (jean), *pul* (pull), *joking* (jogging), *jebba*, *jelbeb*, *جلباب*, *كسوة*... L'**ordre des règles compte** : « veste en jean » doit être une veste, « short en jean » un short, « chaussettes pour sneakers » ou « portefeuille jean » un accessoire, et « short basket » un short (basket-ball), pas une paire de baskets.
- **Couleur principale** : couleur citée dans le nom, sinon la première variante de couleur. Des codes comme `T00849-NOIRDEL` sont aussi reconnus. Le mot « jean » ne sert de couleur (bleu) qu'en dernier recours, car un « Jean » vendu en variante NOIR est noir.
- **Tailles** : seulement les tailles **en stock** quand la boutique donne le stock par taille.
- **Prix** : seuls les prix **en TND** sont conservés (la devise est vérifiée pour chaque plateforme).

- **Ville** : vérifiée sur le site (page contact, page « nos magasins », fiche Shopify), par exemple VOG → Bizerte, LYOUM → La Marsa, UNJA → Sfax, MAGIC → 9 villes, Joliesse → 10 villes (20 magasins), Strass → 7 villes (13 magasins). **Laissée vide** quand le site ne donne pas d'adresse.

3.5 Filtres appliqués pendant la collecte

Filtre	Exemples retirés	Raison
Hors mode	Livres et chapelets (FEModest), valises (Joliesse), coussins	Ce ne sont pas des vêtements
Enfants / bébés	« Sandale fillette », « Sortie naissance », « Baskets Garçonnet »	DressMe s'adresse à la garde-robe d'adultes
Marques étrangères	Nike, Adidas, UGG, Saucony, Moncler… et les noms de modèles (Air Force, Samba…)	Objectif BO6 : promouvoir les marques et boutiques **locales**
Produit sans prix ou sans photo	Photos impossibles à télécharger	Un produit sans photo ne peut pas être affiché ni encodé

Le filtre des marques étrangères est appliqué **avant** le quota par boutique. *Pourquoi :* sinon une boutique comme Moudda aurait perdu une partie de ses 300 places au lieu de les remplir avec des pièces locales.

Équilibre du catalogue. On garde au maximum **300 produits par boutique**, choisis à tour de rôle dans chaque catégorie. *Pourquoi :* sans cela, une grande boutique remplirait le catalogue avec un seul type de produit.

3.6 Photos des produits

Une photo est téléchargée pour chaque produit dans `ml/data/shops/images/`, convertie en JPEG de 800 px maximum (**2 578 photos, 94 Mo**). Le nom du fichier est dérivé de l'URL du produit, il ne change donc pas d'une mise à jour à l'autre, et une photo déjà présente n'est pas retéléchargée. *Pourquoi :* FashionCLIP a besoin des images en local, et l'application les affiche sans dépendre des sites des boutiques (route `/m07/images/…`).

3.7 Résultat du scraping

Boutique	Produits lus	Retenus	Principales exclusions
Moudda	1 250	300	272 marques étrangères, 71 enfants
Manuella	700	300	73 enfants
VOG	1 200	300	—
Exist	1 163	300	12 hors mode
HA	791	300	29 enfants
Strass Accessoires	825	300	30 hors mode, 9 enfants
Joliesse	357	300	18 hors mode (valises), 20 enfants
MAGIC	94	92	2 enfants
NOON Clothing	90	90	—
LYOUM	115	80	33 hors mode (maison, foutas)
Apparel	83	51	17 hors mode, 12 enfants

Boutique	Produits lus	Retenus	Principales exclusions
Corella Store	56	48	6 enfants
UNJA	36	36	—
FEModest	97	27	70 hors mode (livres, chapelets)
Shoes Mall	10	9	—
Scarpa, Sneakers Tunisie, Sneakers Outlet	739	47	687 marques étrangères (les 47 restants aussi, retirés au nettoyage)
Total		**2 578 avec photo**	

3.8 Ajouter ou mettre à jour des boutiques

L'option `--merge` scrape seulement les boutiques demandées et garde les autres telles quelles, avec leurs photos :

```
python -m modules.m07_local_catalogue.scraper.run --shops strass joliesse --merge
```

Pourquoi : ajouter Strass et Joliesse n'a demandé que ces deux boutiques (environ 15 minutes), au lieu de relancer les 13 autres.

4. Nettoyage des données

Le script `scraper/clean.py` transforme `local_catalogue.csv` (2 578 produits) en `local_catalogue_clean.csv` (**2 471 produits**). Chaque étape est comptée dans `ml/data/shops/cleaning_report.md`. Les règles de catégories sont réappliquées au nom de chaque produit : une correction de règle profite au catalogue sans refaire le scraping.

4.1 Prix convertis en nombres et vérifiés en TND

Tous les prix sont des nombres décimaux, strictement positifs. La devise est garantie à la source : les adaptateurs ne gardent que les prix en TND (devise Shopify, champ `currency_code` de WooCommerce, `priceCurrency` du JSON-LD). Une colonne `currency = TND` est ajoutée. Les prix très éloignés de la médiane de leur sous-catégorie (moins d'un cinquième ou plus de cinq fois) sont **signalés** dans `quality_flags` (20 très bas, 5 très hauts) mais **jamais modifiés**. *Pourquoi :* un prix inhabituel n'est pas forcément faux (une épingle de hijab à 0,5 TND est réelle).

4.2 Dernier contrôle « local uniquement »

Le nettoyage repasse le filtre des marques étrangères avec une liste enrichie de noms de modèles vendus sans la marque dans le titre (« AIR FORCE 1 '07 », « BASKET BOY AIR-FORCES », « On Cloudtilt »…). **58 produits retirés**, ainsi que 3 articles enfants et 1 valise.

4.3 Nomenclature commune des catégories

Les catégories sont alignées sur le **contrat de données partagé** du projet (`GarmentCategory` : `top`, `bottom`, `dress`, `outerwear`, `shoes`, `accessory`, `traditional`), avec une **sous-catégorie** précise et un **libellé français** pour l'affichage :

Avant (texte des boutiques)	`category`	`subcategory`	`label_fr`
sneakers, baskets, basket, Samba…	shoes	sneakers	Baskets
chaussures, mocassins, derbies	shoes	shoes	Chaussures
costume, tailleur, smoking	dress	suit	Costumes
collier, pendentif, chaîne	accessory	necklace	Colliers
bracelet, gourmette	accessory	bracelet	Bracelets
bague, alliance	accessory	ring	Bagues
boucles / puces d'oreilles	accessory	earrings	Boucles d'oreilles
jebba, kaftan, جلباب	traditional	traditional	Tenues traditionnelles

Pourquoi : le module Vision (M1) classe la garde-robe avec ces mêmes catégories : M7 et M1 « parlent la même langue ». Les bijoux sont séparés par type pour qu'une recherche « bracelet » ne renvoie pas de bagues. Répartition finale : chaussures 688, accessoires 673, hauts 408, bas 353, robes/ensembles/costumes 205, vestes/manteaux 131, tenues traditionnelles 13.

4.4 Couleurs, tailles et villes uniformisées

- **Couleurs** : un code anglais (`white`, `navy`…) identique à celui du module Vision, plus le nom français (`Blanc`) et un code hexadécimal (`#FFFFFF`).
- **Tailles** : une seule notation, triée. `2XL` devient `XXL`, `ST` devient `Taille unique`, les tailles sont dans l'ordre (`S, M, L, XL`, `38, 39, 40`). Une colonne `size_type` indique le système (`letter`, `numeric`, `one_size`, `bra`…). On partait de **94 notations différentes**.
- **Villes** : nom officiel (`Béja`, `La Marsa`) et **gouvernorat** (`La Marsa` → `Tunis`). *Pourquoi :* une utilisatrice qui cherche « à Tunis » doit aussi voir les boutiques de La Marsa.

4.5 Suppression des doublons

Comparer seulement le nom et le prix aurait été une erreur : des produits différents portent souvent le même nom générique (« Basket Homme » à 59,5 TND existe en 4 couleurs avec 4 photos différentes). Deux produits sont donc des doublons s'ils ont **le même nom, le même prix et une photo quasi identique**, comparée avec une **empreinte perceptuelle** (*dHash* sur 64 bits). On garde l'exemplaire en stock avec le plus de tailles. **45 doublons retirés** : 13 dans une même boutique et 32 entre deux boutiques (par exemple Corella et Apparel vendent les mêmes articles au même prix).

4.6 Vérification des URL

Chaque URL de fiche produit et de photo a été vérifiée par une requête HTTP (**4 944 URL**).

- Une fiche qui renvoie 404/410, ou qui redirige vers la page d'accueil, est **morte** : le produit est retiré.
- Les autres échecs sont **signalés** sans retirer le produit.
- Les résultats sont mis en cache (`url_checks.json`) : une relance ne vérifie que les nouvelles URL.

Au premier passage, 625 fiches Shopify ont répondu « 429 » (limite de débit). Ce n'est pas une page morte : elles ont été revérifiées plus lentement (1 requête/s, en respectant l'en-tête `Retry-After`). **Résultat : 4 944 URL sur 4 944 répondent correctement (200), aucun produit mort.**

4.7 Informations manquantes marquées, jamais inventées

Une valeur absente reste **vide** et le nom du champ est listé dans la colonne `missing_fields`. Aucune valeur n'est devinée.

Champ	Renseigné	Pourquoi il manque parfois
Couleur	85,6 %	Nom du produit sans couleur et pas de variante de couleur (bijoux, accessoires…)
Taille	62,1 %	Accessoires et bijoux (sans taille), ou boutique qui ne publie pas les tailles
Ville	68,6 %	Boutiques en ligne sans adresse publiée (NOON, Exist, HA…)

Pourquoi : une couleur ou une ville inventée pourrait envoyer l'utilisatrice dans une boutique qui n'a pas l'article. Le moteur en tient compte (§5.5) et le dit (« Couleur non précisée par la boutique », « Ville non précisée »).

4.8 Identifiants stables

Chaque produit reçoit un identifiant `prd_` + empreinte de son URL (par exemple `prd_2b7db627f3`). *Pourquoi :* la numérotation P0001… change à chaque scraping, alors que d'autres modules (favoris, liens partenaires M13, partage M10) ont besoin d'une référence durable.

4.9 Le catalogue final

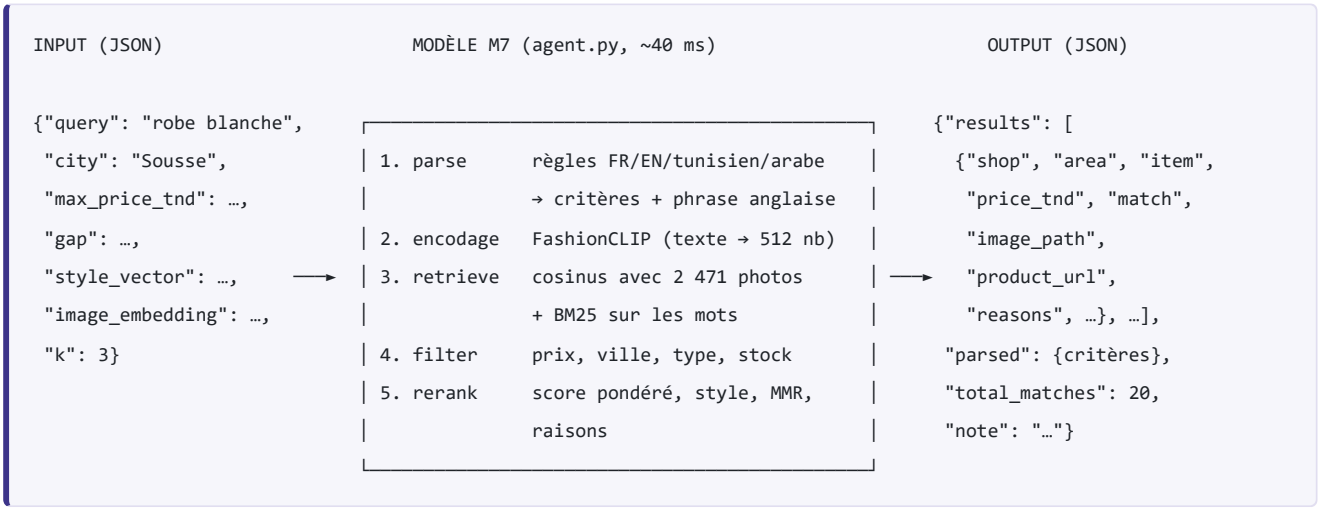
Indicateur	Valeur
Produits	**2 471**
Boutiques	**15**
Marques tunisiennes / boutiques locales multimarques	2 153 / 318
En stock	2 060
Prix médian	69,9 TND (de 0,5 à 399,99 TND)
Bijoux	38 colliers, 27 bagues, 12 bracelets, 10 paires de boucles d'oreilles

Boutique	Produits	Boutique	Produits	Boutique	Produits
Manuella	300	Moudda	282	Corella Store	48
HA	299	MAGIC	90	UNJA	36
Joliesse	299	NOON Clothing	90	FEModest	27
Strass Accessoires	299	LYOUM	80	Apparel	18
VOG	299	Exist	295	Shoes Mall	9

5. Le module M7

5.0 Input → modèle → output de M7

Le §2.1 suit tout le parcours, du scraping à la recherche. Ici on ne regarde que **le module M7 lui-même**, c'est-à-dire ce qui se passe à chaque appel de `POST /m07/search` (ou de l'outil `localshop`) : le catalogue et l'index sont déjà prêts.



Partie	Ce qu'elle reçoit	Ce qu'elle fait	Ce qu'elle produit
Input	Une phrase (`query`), un manque de la garde-robe (`gap`), des filtres explicites (`city`, `max_price_tnd`, `subcategory`, `color` …), éventuellement un vecteur de style (M5) ou d'image (M1)	Validation Pydantic (`LocalShopInput`) : au moins un critère de recherche	Une demande valide, sinon erreur 422
Modèle	La demande + le catalogue + l'index	Règles (compréhension, filtres) + **FashionCLIP** (seule partie apprise) + BM25 + score pondéré	Les produits classés
Output	Les k meilleurs produits	`LocalShopOutput`	`results` (boutique, ville, article, prix, `match`, photo, lien, raisons), `parsed` (ce qui a été compris), `total_matches`, `note`

Exemples réels (catalogue actuel, 15 boutiques)

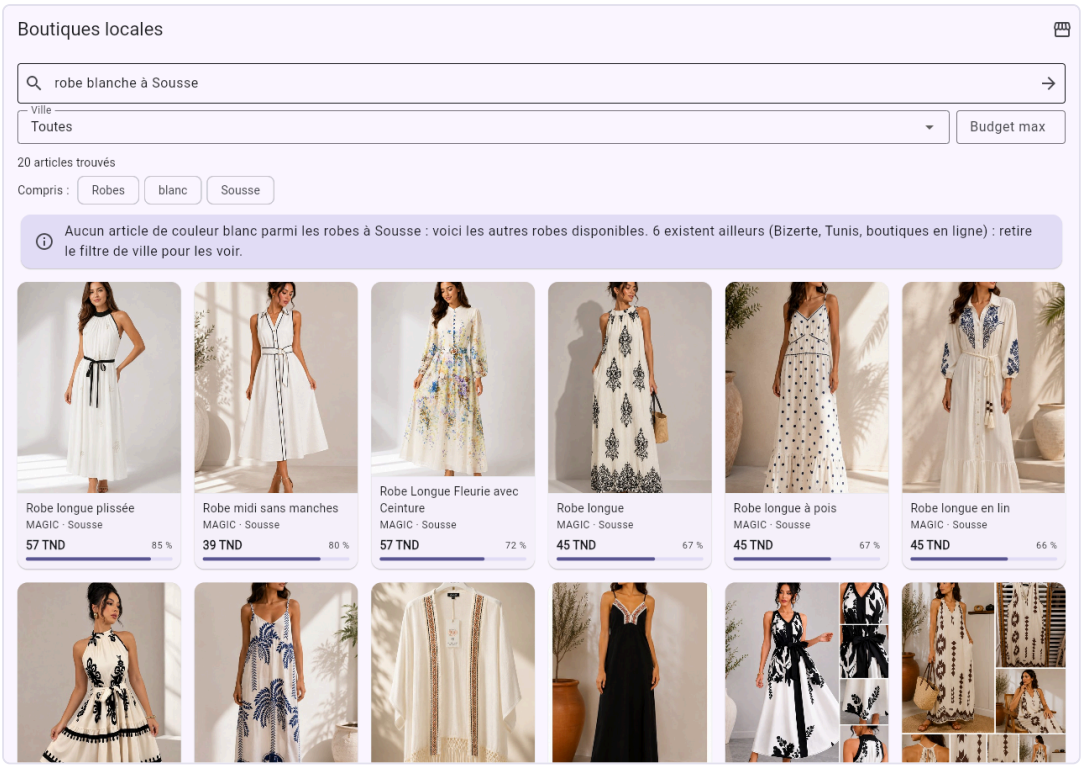
Chaque ligne est une vraie réponse de l'API, avec `k = 3` (les 3 premiers résultats).

Input envoyé à M7	Ce que M7 a compris (`parsed`)	Output (`total_matches` puis résultats)
`{"query": "baskets blanches à Tunis pour un budget de 80 TND"}`	baskets · blanc · Tunis · ≤ 80 TND ; prompt « a photo of white sneakers »	**16** — Manuella · Tunis · Basket femme · 62,65 TND (0,882) ; Manuella · Tunis · Basket à plateforme femme · 69,3 TND (0,874) ; Joliesse · Tunis · Baskets Basses Effet Serpent · 79,99 TND (0,761)

Input envoyé à M7	Ce que M7 a compris (`parsed`)	Output (`total_matches` puis résultats)
`{"query": "robe blanche", "city": "Sousse"}`	robe · blanc · Sousse	**20** — MAGIC · Sousse · Robe longue plissée · 57 TND (0,853) ; Robe midi sans manches · 39 TND (0,803) ; Robe Longue Fleurie · 57 TND (0,715). *note* : « Aucun article de couleur blanc parmi les robes à Sousse : voici les autres robes disponibles. 6 existent ailleurs (Bizerte, Tunis, boutiques en ligne) : retire le filtre de ville pour les voir. »
`{"query": "colier"}` (faute de frappe)	collier ; `corrected` = « collier »	**37** — Strass · Collier avec Pendentifs Trèfles · 22,9 TND (0,944) ; Collier avec Étoile de Mer · 39,9 TND (0,94) ; Collier Sautoir · 79,9 TND (0,93)
`{"gap": "white shirt", "city": "Tunis"}` (Smart Buy)	chemise · blanc · Tunis	**13** — Moudda · Chemise Satin · 69,9 TND (0,771) ; MAGIC · Chemise brodée · 50 TND (0,696) ; CHEMISE W RRAYE · 59,99 TND (0,619). *note* : aucune chemise blanche à Tunis, 11 ailleurs (Bizerte, boutiques en ligne)
`{"subcategory": "bag", "color": "camel", "max_price_tnd": 150}` (champs seuls, sans phrase)	sac · camel · ≤ 150 TND ; prompt « a photo of camel handbag »	**343** — Manuella · Sac à main minimaliste · 129 TND (0,955) ; Joliesse · Sac à Main Structuré · 139,99 TND (0,923) ; Manuella · Sac pochette · 69,3 TND (0,9)
`{"query": "zorglub"}` (mot inconnu)	rien	**0** — *note* : « Je n'ai trouvé aucun article correspondant à « zorglub » dans les boutiques locales. »
`{"city": "Tunis"}` (rien à chercher)	—	Erreur **422** : l'entrée est refusée avant le modèle

À retenir.

- La **couleur** est un critère souple : si aucune pièce de la bonne couleur n'existe dans la ville demandée, M7 montre les autres pièces du bon type **et le dit** dans `note`, au lieu d'inventer ou de renvoyer une page vide.
- Le champ `area` donne **la ville du magasin qui correspond au filtre**. Pour une chaîne présente dans plusieurs villes (MAGIC : Tunis, Sousse, Sfax…), une recherche à Sousse affiche « MAGIC · Sousse ».
- Le `match` (0 à 1) et les `reasons` sont calculés uniquement à partir des données et des scores.



5.1 Architecture et contrat

Le module suit la structure des modules partagés (`backend/modules/m07_local_catalogue/` avec `contract.py`, `router.py`, `service.py`, `mock.py`, `tool.py`, `tests/`), plus `agent.py` et `pipeline/`, et le pipeline prévu par l'architecture : **parse** → **retrieve** → **filter** → **rerank**.

Entrée (`LocalShopInput`) — au moins un des champs `query`, `gap`, `category` / `subcategory` ou `image_embedding` :

Champ	Exemple	Utilisé par
`query`	« baskets blanches à Tunis pour 80 TND »	Chat (BO8), écrans de recherche
`gap`	« white shirt »	Smart Buy (BO5), analyse de la garde-robe
`city`, `max_price_tnd`, `min_price_tnd`	« Tunis », 80	Filtres explicites
`category`, `subcategory`, `color`	shoes, sneakers, white	Filtres explicites
`user_id`, `style_vector`	« usr_042 », « vec_u042 » (ou 512 nombres)	Profil de style (M5) : l'identifiant est demandé à M5, comme dans l'architecture
`image_embedding`	512 nombres	Photo d'une pièce (Vision M1 / Smart Buy)
`include_unavailable`, `k`	false, 10	Options

Les champs explicites ont priorité sur ce qui est lu dans le texte, et le prompt FashionCLIP est reconstruit à partir des critères finaux.

Sortie (`LocalShopOutput`) : la liste `results` reprend les champs prévus par l'architecture (`shop`, `area`, `item`, `price_tnd`, `match`), enrichis de l'identifiant, des liens, de la photo, des tailles, du stock et des **raisons**. S'y ajoutent `parsed` (les filtres réellement appliqués et la requête corrigée), `total_matches` et `note` (ville relâchée,

articles en rupture, rien trouvé…).

5.2 Index FashionCLIP (base de connaissances)

FashionCLIP (`patrickjohncyh/fashion-clip`) est un modèle d'intelligence artificielle : un réseau de neurones (architecture CLIP) entraîné par des chercheurs sur environ 800 000 photos de mode avec leur description. Il transforme **une photo en une liste de 512 nombres** (un « vecteur ») et **une phrase en anglais en une autre liste de 512 nombres**, de sorte qu'une photo et sa bonne description aient des vecteurs proches : *« a photo of white dress »* est proche des photos de robes blanches. Nous ne l'avons pas réentraîné : il est utilisé tel quel (*zero-shot*).

Pourquoi FashionCLIP :

- il comprend le **contenu visuel** (forme, couleur, motif) même quand la description est pauvre ou fausse ;
- l'architecture le recommande pour le module Vision : M7, M1 (garde-robe) et M5 (vecteur de style) partagent le même espace ;
- il tourne **en local** sur CPU, sans service payant.

Le script `pipeline/ingest.py` encode chaque photo du catalogue (`ml/data/shops/index/`, 512 nombres par photo). La première construction a pris **3 min 42 s pour 1 874 photos** sur CPU. L'index est **incrémental** : à l'ajout de Strass et Joliesse, seules les **599 nouvelles photos** ont été encodées, en **35 s**. Le modèle (581 Mo) est téléchargé au premier lancement, ou lu depuis `ml/models/fashion-clip/` s'il y est (DVC).

5.3 Compréhension de la requête (`pipeline/parse.py`)

La phrase de l'utilisatrice est transformée en critères :

```
« Je cherche une chemise en lin blanche moins de 100 dt à la Marsa »
  → subcategory: shirt · color: white · max_price_tnd: 100 · city: La Marsa (gouvernorat Tunis)
  → prompt FashionCLIP : « a photo of white linen shirt »
```

Il reconnaît les budgets (« moins de 80 dt », « entre 50 et 120 dinars »), les villes tunisiennes, les couleurs, les catégories et des attributs (lin → linen, cuir → leather, fleuri → floral…).

Correction des fautes de frappe. Un mot inconnu mais très proche d'un mot connu est corrigé (« colier » → « collier »). Le vocabulaire de référence réunit les mots-clés des règles et les mots réellement utilisés par les boutiques. L'application affiche « Recherche corrigée : « collier » ».

Pourquoi un analyseur à règles et non un LLM : il utilise **exactement les mêmes listes de mots** que le nettoyage (« baskets » dans une requête et dans un produit tombent toujours dans la même sous-catégorie), il est instantané, gratuit, déterministe et testable.

Pourquoi un prompt en anglais : FashionCLIP n'a été entraîné que sur de l'anglais (§5.10). La requête française est reformulée en anglais à partir des critères compris. **Les autres langues (français, tunisien, arabe) sont donc gérées par nos règles, pas par FashionCLIP.**

5.4 Recherche hybride (`pipeline/retrieve.py`)

Deux scores sont calculés pour **chaque** produit :

- **Visuel (FashionCLIP)** : similarité cosinus entre le prompt (ou la photo envoyée) et la photo du produit ;

- **Texte (BM25)** : algorithme classique de recherche par mots (statistiques, sans apprentissage) sur le nom, la catégorie, la couleur, la boutique et la ville.

Pourquoi hybride : le visuel trouve des pièces même mal décrites ; le texte capte les mots précis (un nom de boutique, « jebba ») que l'image ne contient pas. *Pourquoi un calcul exact :* avec ~2 500 produits, comparer à tous prend quelques millisecondes ; une base vectorielle (pgvector, Qdrant) n'est utile qu'à plus grande échelle.

5.5 Filtres (`pipeline/filter.py`)

Filtre	Type	Raison
Prix ≤ budget, prix ≥ minimum	Strict	Le budget en TND est une contrainte de l'utilisatrice
Ville **ou son gouvernorat**	Strict	« Tunis » inclut La Marsa ; une boutique sans adresse n'est pas mise dans une ville au hasard
Catégorie / sous-catégorie	Strict	Une demande de baskets ne doit pas renvoyer de sandales
En stock	Strict (désactivable)	Ne pas proposer ce qu'on ne peut pas acheter
Couleur	**Souple** (dans le score)	14 % des produits n'ont pas de couleur connue : un filtre strict les éliminerait à tort

Trois garde-fous complètent ces filtres :

- **Rien de compris** (ni type, ni couleur, ni photo) : on ne garde que les produits qui contiennent les mots cherchés, sinon le moteur répond « Je n'ai trouvé aucun article correspondant à … ». Il ne remplit plus la liste avec des pièces sans rapport.
- **Ville sans résultat** : le filtre de ville est relâché et `note` l'indique.
- **Rien en stock** : les articles en rupture sont montrés avec la mention « Rupture de stock », plutôt qu'une page vide.

5.6 Score, style, diversité et explications (`pipeline/rerank.py`)

Le score `match` (entre 0 et 1) est une moyenne pondérée des éléments qui s'appliquent :

Composante	Poids	Mesure
Visuel	0,45	FashionCLIP requête ↔ photo, ramené sur 0–1
Texte	0,20	BM25, relatif au meilleur candidat
Attributs	0,20	Sous-catégorie demandée ; couleur : identique = 1, inconnue = 0,5, autre = 0
Style	0,15	Vecteur de style de l'utilisatrice ↔ photo du produit

Les poids sont **renormalisés** selon les composantes disponibles.

Calibration. Les similarités FashionCLIP sont faibles en valeur absolue. On a mesuré leur distribution sur le vrai catalogue : texte → photo (médiane ≈ 0,11, 99e centile ≈ 0,30), d'où la plage [0,15 ; 0,33] ; photo → photo (médiane ≈ 0,42, 99e centile ≈ 0,66), d'où la plage [0,40 ; 0,85].

Diversité (MMR). On évite de renvoyer 5 photos presque identiques (le même modèle en 5 coloris).

Explications sans invention. Chaque résultat a des raisons construites uniquement à partir des données et des scores : « Baskets », « Couleur : blanc », « 69.3 TND (budget 80 TND) », « Boutique à Tunis (près de La Marsa) », « Ressemble visuellement à la demande », « Marque tunisienne ». Elles alimentent l'écran « Pourquoi cette pièce locale » (BO6) et le module LLM (M8) sans qu'un modèle puisse inventer un fait.

5.7 Intégration dans le projet

- **Outil de l'orchestrateur** : `tool.py` expose l'agent sous le nom `localshop`. Il est découvert automatiquement par `orchestrator/registry.py`.
- **API REST** (`router.py` , inclus automatiquement par `api/main.py` sous le préfixe `/m07`) :
  - `POST /m07/search` : recherche ;
  - `GET /m07/shops` : liste des boutiques ;
  - `GET /m07/products/{product_id}` : fiche d'un produit ;
  - `GET /m07/images/{nom}` : photo locale d'un produit.
- **Sans les données** sur la machine (clone sans `dvc pull`), le service répond avec `mock.py` au lieu d'échouer : les modules qui appellent M7 ne sont jamais bloqués.
- **Écran BO6** (`mobile/lib/features/bo6_local_market/`) : recherche, filtres ville et budget, critères compris, grille de photos, fiche produit, liste des boutiques (§6.4).
- Le modèle est chargé à la première recherche (environ 12 s), puis **chaque recherche prend environ 40 ms**.

5.8 Exemples de résultats réels

« baskets blanches à Tunis pour un budget de 80 TND » → 16 produits (Manuella et Joliesse) :

match	Boutique	Ville	Article	Prix
0,88	Manuella	Tunis	Basket femme	62,65 TND
0,87	Manuella	Tunis	Basket à plateforme femme	69,3 TND
0,76	Joliesse	Tunis	Baskets Basses Effet Serpent à Détails Dorés	79,99 TND

Le premier produit est étiqueté « gris » par la boutique, mais sa photo montre une basket crème/blanche : FashionCLIP le classe donc haut, et la raison affichée signale l'étiquette de la boutique.

Requête	Résultat
« une robe blanche »	78 robes, robes blanches en premier (HA 41,99 TND, Moudda 45,43 TND…)
« collier doré »	37 colliers, surtout Strass Accessoires (34,9 – 39,9 TND)
« bracelet »	12 bracelets (Strass, Exist)
« sandales à Sfax »	97 sandales, Joliesse en premier (2 magasins à Sfax)
« costume »	6 costumes complets (Exist, 249,99 – 399,99 TND)
« jebba brodée »	13 tenues traditionnelles (UNJA, Corella, FEModest)

5.9 Corrections après les premiers tests dans l'application

Les tests dans l'application ont révélé trois problèmes, venant tous de la compréhension de la requête (et non de FashionCLIP) :

Recherche	Avant	Cause	Après
« costume »	Costumes puis bracelets, casquettes, pulls	Mot inconnu + liste toujours remplie, même sans rapport	6 costumes seulement
« colier » (faute)	Liste au hasard	Pas de tolérance aux fautes	Corrigé en « collier » : 37 colliers
« bracelet »	1 bracelet + 4 bagues	Tous les bijoux dans une seule catégorie	12 bracelets seulement

Corrections : vocabulaire des costumes, bijoux séparés par type, correction des fautes, garde-fou « rien compris », articles en rupture montrés avec mention. Chaque cas a un test automatique.

Un deuxième retour a porté sur le **filtre des villes** et l'affichage :

Problème signalé	Cause	Correction
« robe blanche » + ville Sousse affichait « MAGIC · Tunis » : le filtre semblait ne pas marcher	Le filtre fonctionnait (seuls les magasins de Sousse étaient gardés), mais la carte affichait la **première** ville de la chaîne (Tunis) ; et il n'existe aucune robe blanche à Sousse	`area` ne contient plus que les villes qui correspondent au filtre (« MAGIC · Sousse ») ; sans filtre, la carte affiche « Tunis +8 » ; `note` explique qu'aucune robe blanche n'est à Sousse et où il y en a
Bandeau « DEBUG » en haut à droite	Bandeau affiché par défaut par Flutter en mode développement	`debugShowCheckedModeBanner: false` dans `mobile/lib/main.dart`

5.10 FashionCLIP est-il le meilleur choix ?

Comparaison sur notre catalogue. Trois modèles de mode ont été évalués sur nos 1 873 photos de l'époque, avec les étiquettes des boutiques comme référence (105 recherches « couleur + type ») :

Mesure	FashionCLIP (retenu)	Marqo-FashionSigLIP	Marqo-FashionCLIP
Recherche « couleur + type » : bons résultats dans le top 10	29,7 %	**32,5 %**	30,9 %
Recherche par type : bons résultats dans le top 10	67,6 %	68,9 %	**69,4 %**
Type reconnu sur la photo	66,8 %	**72,0 %**	67,3 %
Couleur reconnue sur la photo	47,9 %	47,5 %	**48,9 %**
Vitesse sur CPU	**96 ms/photo**	234 ms/photo	236 ms/photo
Taille du vecteur	**512**	768	**512**

Décision : garder FashionCLIP pour l'instant. Les modèles récents gagnent seulement 1 à 3 points en recherche (proche du bruit de mesure, avec des étiquettes imparfaites) ; dans M7 le visuel ne pèse que 45 % du score et les filtres garantissent l'essentiel ; FashionCLIP est 2,4 fois plus rapide et respecte la convention de 512 du projet. Marqo-FashionSigLIP (768) imposerait de changer aussi M1 et M5 : c'est une décision d'équipe.

Langues. FashionCLIP ne comprend vraiment que l'anglais. Sur 12 recherches envoyées directement au modèle, le bon type apparaît dans le top 10 pour 77 % des requêtes en anglais (« black skirt ») contre 58 % en français (« jupe noire » : 0 %). C'est pourquoi le français, le tunisien et l'arabe passent par nos règles, qui reformulent en anglais.

Faut-il réentraîner (fine-tuning) ? Pas en premier : les erreurs observées venaient de la compréhension de la requête. Ordre recommandé : (1) un jeu de test de 50 à 100 requêtes réelles annotées pour mesurer ; (2) la compréhension par le LLM de M8 pour les phrases inhabituelles ; (3) les photos détourées de M1 ; (4) seulement ensuite un fine-tuning léger (dernières couches ou LoRA, sur GPU Colab) avec nos paires photo + nom français et des données tunisiennes étiquetées, en partageant le modèle obtenu avec M1 et M5.

6. Comment tester le module

Les commandes backend se lancent depuis `backend/`, celles de l'application depuis `mobile/`.

6.1 Tests automatiques (sans modèle ni internet)

```
cd backend
pip install -r requirements-dev.txt
pytest modules/m07_local_catalogue -q     # résultat attendu : 54 passed
pytest -q                                 # tout le backend : 79 passed
```

Ces tests utilisent un mini-catalogue et un faux encodeur : ils n'ont besoin ni de `torch`, ni du modèle, ni des données.

Fichier	Ce qui est vérifié
`test_contract.py`	Validation des entrées, agent léger à la création
`test_mock.py`	La réponse fixe de `mock.py` respecte le contrat
`test_router.py`	Routes `/m07/*`, repli sur le mock sans données, photos protégées, vecteur de style demandé à M5
`test_search.py`	Exemple « baskets blanches à Tunis 80 TND », gouvernorat, ville relâchée, champs explicites, couleur inconnue non devinée, style et photo, fautes de frappe, bijoux et costumes, mots inconnus, articles en rupture, ville affichée selon le filtre et note de couleur absente
`test_ingest.py`	Index incrémental : seules les nouvelles photos sont encodées
`test_clean.py`	Tailles, doublons, statut des URL, nomenclature commune, villes
`test_scraper_normalize.py`	Catégories (FR/EN/tunisien/arabe), couleurs, tailles, marques, équilibrage

Front : `cd mobile && flutter test` → **4 tests**.

6.2 Prérequis pour une recherche réelle

```
pip install -r modules/m07_local_catalogue/requirements.txt   # torch, transformers, requests, pillow...
```

Les données du module sont dans `ml/data/shops/` et partagées avec DVC (`dvc pull`). Sans DVC, on les reconstruit :

```
python -m modules.m07_local_catalogue.scraper.run      # 1. scraping + photos          (~30 min)
python -m modules.m07_local_catalogue.scraper.clean    # 2. nettoyage + vérif. des URL (~15 min la 1re fois)
python -m modules.m07_local_catalogue.pipeline.ingest  # 3. index FashionCLIP          (~5 min la 1re fois, puis
incrémental)
```

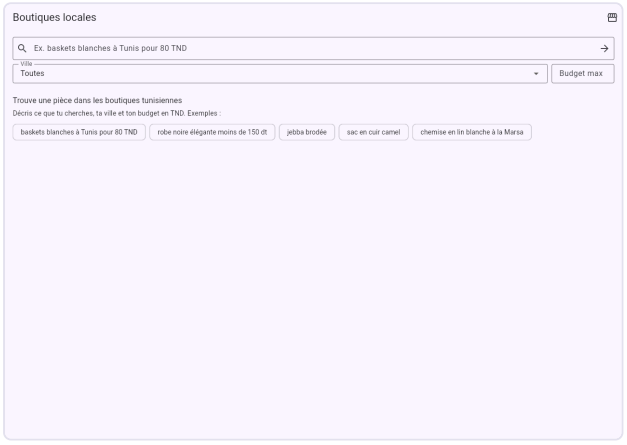
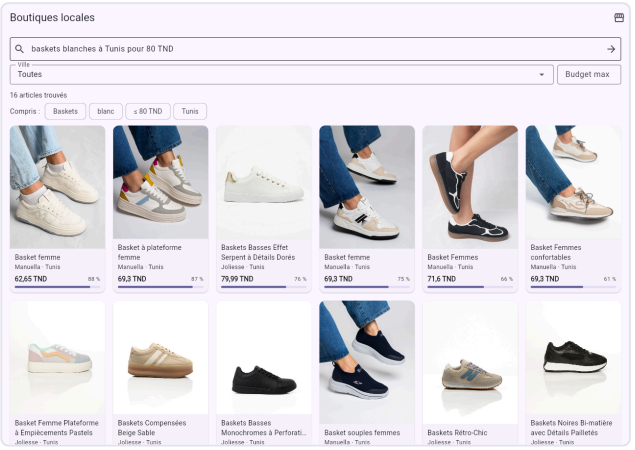
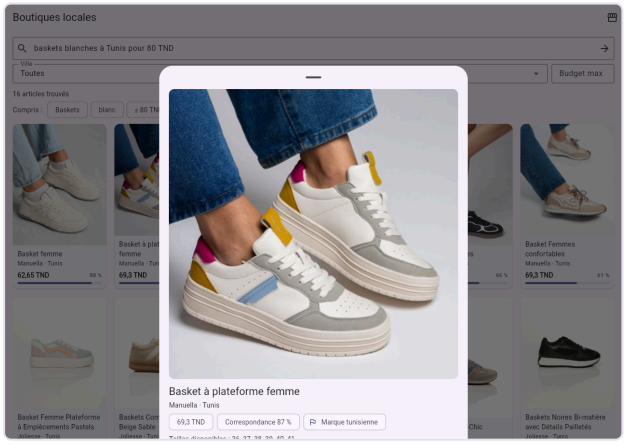
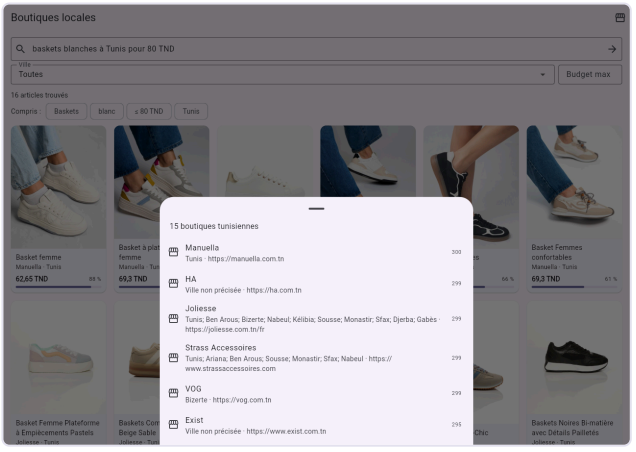
6.3 Tester par l'API (Swagger)

```
cd backend
uvicorn api.main:app --port 8010
```

Ouvrir **http://127.0.0.1:8010/docs**, choisir `POST /m07/search`, **Try it out**, coller un corps JSON (voir le tableau du §6.4), **Execute**. Le port 8000 est déjà utilisé par une autre application sur la machine de développement ; n'importe quel port libre convient.

6.4 Tester dans l'application (front Flutter)

L'écran BO6 « Boutiques locales » (`mobile/lib/features/bo6_local_market/`) appelle M7 par l'API, comme le fera l'application finale.

Accueil	Résultats
	
Fiche produit	**Boutiques**
	

1. **Lancer l'API** (terminal 1) : `cd backend` puis `uvicorn api.main:app --port 8010`
2. **Lancer l'application** (terminal 2) :

```
$env:Path += ";C:\Users\Kinza\flutter\bin"
cd mobile
flutter pub get
flutter create . --platforms=web     # une seule fois : génère web/ (ne pas le commiter, c'est M6 qui génère les plateformes)
flutter run -d web-server --web-port 8081 --dart-define=API_URL=http://localhost:8010
```

3. **Ouvrir Chrome** sur l'écran BO6 (terminal 3) :

```
& "C:\Program Files\Google\Chrome\Application\chrome.exe" --disable-web-security --user-data-dir="$env:TEMP\chrome-dressme-test" "http://localhost:8081/#/bo6_local_market"
```

4. Taper une demande (ou cliquer sur un exemple), puis toucher un article pour ouvrir sa fiche.

Action	Résultat attendu
« baskets blanches à Tunis pour 80 TND »	16 baskets (Manuella, Joliesse) ≤ 80 TND ; « Compris : Baskets · blanc · ≤ 80 TND · Tunis »
« une robe blanche »	78 robes, 24 affichées, robes blanches en premier, avec boutique, ville et prix
« robe blanche » + Ville : Sousse	20 robes « MAGIC · Sousse » et la note « Aucun article de couleur blanc parmi les robes à Sousse… 6 existent ailleurs »
« colier » (avec la faute)	« Recherche corrigée : « collier » », 37 colliers
« bracelet » / « costume »	12 bracelets / 6 costumes, sans autre type d'article
`{"subcategory": "bag", "color": "camel", "max_price_tnd": 150}` (API)	Sacs, camel en premier
`{"gap": "white shirt", "city": "Tunis"}` (API)	Chemises de boutiques à Tunis, classées par ressemblance avec une chemise blanche
Toucher un article	Photo, prix, correspondance, tailles, raisons, lien de la fiche produit à copier
Icône boutique (en haut à droite)	Les 15 boutiques avec leurs villes et leur site
`{"city": "Tunis"}` (API)	Erreur **422** (rien à chercher)

Points d'attention :

- **Ne pas utiliser le Chrome habituel** : la page s'affiche, mais les recherches échouent (« Serveur injoignable ») car le navigateur bloque les appels vers un autre port (règle CORS) et l'API n'a pas encore de configuration CORS (`api/main.py`, fichier du responsable). Le drapeau `--disable-web-security`, réservé aux tests en local, lève ce blocage. Sur téléphone ou émulateur, il n'est pas nécessaire : `flutter run --dart-define=API_URL=http://<IP-du-PC>:8010`.
- **Éviter le port 5060** pour l'application : Chrome le refuse (`ERR_UNSAFE_PORT`, port réservé à la téléphonie SIP).

6.5 Tester directement en Python

```
import asyncio
from modules.m07_local_catalogue.agent import LocalShopAgent

agent = LocalShopAgent()
out = asyncio.run(agent({"query": "baskets blanches à Tunis pour un budget de 80 TND", "k": 5}))
print(out["parsed"])              # critères compris
for r in out["results"]:
    print(r["match"], r["shop"], r["area"], r["item"], r["price_tnd"], r["reasons"])
```

6.6 Vérifier les données

- Ouvrir `ml/data/shops/local_catalogue_clean.csv` dans Excel (encodage UTF-8).
- Lire `ml/data/shops/cleaning_report.md` pour le détail de chaque étape du nettoyage.
- Filtrer la colonne `quality_flags` pour voir les prix à contrôler.

7. Limites et pistes d'amélioration

Limite	Piste
31 % des produits sans ville (boutiques en ligne sans adresse)	Contacter les boutiques ; ajouter une notion de « livraison partout »
Boutiques sans site (HK-STORE, GAŸA, The Queen Accessories, Azar Shoes…)	Fichier partenaire (CSV) rempli à la main ou par la boutique, comme le prévoit l'architecture
Mots absents du vocabulaire non compris (le moteur le dit au lieu d'inventer)	Compréhension par le LLM de M8 pour les phrases inhabituelles
FashionCLIP ne comprend que l'anglais	Règles actuelles ; tester un modèle multilingue (jina-clip-v2) avec le script de comparaison
Pas encore d'évaluation sur de vraies requêtes annotées	Créer 50 à 100 requêtes annotées et mesurer dans `ml/notebooks/localcatalogue/`
Recherche par photo influencée par le style de photo (même studio)	Préciser `subcategory` ; utiliser l'image détourée de M1
Plage du vecteur de style non calibrée (pas encore de profils M5)	La recalibrer quand M5 fournira des vecteurs réels
Ad. Mall, boutique.tn, D'Stock non couverts	Écrire des analyseurs HTML dédiés
Données figées entre deux scrapings	Planifier la relance (Celery beat, M4) avec `--merge`

8. Annexe : fichiers du module

```
backend/modules/m07_local_catalogue/
├── agent.py            agent M7 : parse → retrieve → filter → rerank
├── contract.py         entrée / sortie (Pydantic)
├── taxonomy.py         vocabulaire commun : catégories, bijoux, couleurs, villes et gouvernorats
├── config.yaml         modèle, poids du score, plages de calibration, emplacement des données
├── paths.py            emplacement des données (ml/data/shops) et du modèle
├── clients.py          appels aux autres modules (M5 : vecteur de style)
├── router.py           routes /m07/* (incluses automatiquement par api/main.py)
├── service.py          implémentation réelle partagée par l'API et l'outil
├── mock.py             réponse fixe au même format, pour les tests des autres modules
├── tool.py             outil pour l'orchestrateur (découvert automatiquement)
├── pipeline/
│   ├── parse.py        compréhension de la requête + correction des fautes
│   ├── retrieve.py     FashionCLIP + BM25
│   ├── filter.py       prix, ville, catégorie, stock
│   ├── rerank.py       score, style, diversité, raisons
│   ├── encoder.py      FashionCLIP en local
│   ├── catalogue.py    chargement de la base + index + vocabulaire
│   └── ingest.py       construction incrémentale de l'index (hors ligne)
├── scraper/
│   ├── shops.py        boutiques retenues et écartées (avec raisons)
│   ├── sources.py      un adaptateur par plateforme
│   ├── client.py       client HTTP respectueux (robots.txt, délais, sans cookies)
│   ├── normalize.py    règles de catégories, couleurs, tailles, marques, enfants
│   ├── images.py       téléchargement des photos
│   ├── run.py          lancement du scraping (--shops, --merge)
│   └── clean.py        nettoyage + vérification des URL
└── tests/              53 tests
ml/data/shops/          données du module (DVC, hors git)
├── local_catalogue.csv / .xlsx   données brutes du scraping
├── local_catalogue_clean.csv     base de connaissances nettoyée
├── cleaning_report.md            compte rendu du nettoyage
└── images/  index/  url_checks.json
mobile/lib/features/bo6_local_market/   écran BO6 « Boutiques locales » (Flutter)
mobile/test/bo6_local_market_test.dart  tests de l'écran
```